

使用 Jib 從 Google App Engine Java 應用程式遷移至 Cloud Run

來源：<https://codelabs.developers.google.com/cloud-gae-java-migrate-jib?hl=zh-tw>

步驟數：7

瞭解如何轉換簡單的 Java App Engine 應用程式、使用 Jib 將應用程式容器化，以及將應用程式移至 Cloud Run

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 修改應用程式檔案](#)
5. [5. 容器化應用程式](#)
6. [6. 摘要/清除](#)
7. [7. 其他資源](#)

1. 總覽

本系列程式碼實驗室 (自學式實作教學課程) 旨在協助 [Google App Engine](#) (標準) Java 開發人員完成一系列遷移作業，進而將應用程式現代化。按照這些步驟操作，即可更新應用程式，使其更具可攜性，並決定是否要將應用程式容器化，以便用於 [Cloud Run](#) (Google Cloud 的容器代管服務，與 App Engine 類似) 和其他容器代管服務。

本教學課程說明如何使用 [Jib](#) 將 App Engine 應用程式容器化，以便部署至 Cloud Run 全代管服務。您可以使用 Jib 建立 [Docker](#) 映像檔。Docker 是業界知名的平台，可開發、推送及執行容器中的應用程式。

除了說明從 App Engine 遷移至 Cloud Run 的必要步驟，我們也會教您如何將 Java 8 App Engine 應用程式升級至 Java 17。

如果應用程式大量使用 App Engine 舊版套裝組合服務或其他 App Engine 功能，建議先遷移這些套裝組合服務或取代這些功能，再移至 Cloud Run。如果您需要更多時間研究遷移選項，或想暫時繼續使用舊版套裝組合服務，升級至較新的執行階段時，可以繼續[存取 Java 11/17 適用的 App Engine 套裝組合服務](#)。應用程式更具可攜性後，請返回本程式碼研究室，瞭解如何將這些指示套用至應用程式。

在接下來的研究室中

- 使用 [Cloud Shell](#)
- 啟用 Cloud Run、Artifact Registry 和 Cloud Build API
- 使用 Jib 和 Cloud Build 將應用程式容器化
- 將容器映像檔部署至 Cloud Run

軟硬體需求

- [Google Cloud Platform 專案](#)，並啟用有效的 [GCP 帳單帳戶](#)和 [App Engine](#)
- 熟悉常見的 Linux 指令
- 具備[開發及部署](#) App Engine 應用程式的基本知識
- 您想遷移至 Java 17 並部署至 Cloud Run 的 Java 8 Servlet 應用程式 (可以是 App Engine 上的應用程式，或只是來源)

問卷調查

2. 背景

App Engine 和 Cloud Functions 等平台即服務 (PaaS) 系統可為團隊和應用程式提供許多便利功能，例如讓系統管理員和 DevOps 專注於建構解決方案。使用無伺服器平台時，應用程式可視需要自動擴充資源，並在用量為零時縮減資源，以按用量計費的方式控管費用，還能使用各種常見的開發語言。

不過，容器的彈性也相當吸引人。容器可讓您選擇任何語言、程式庫和二進位檔，兼具無伺服器架構的便利性與容器的彈性。這就是 [Cloud Run](#) 的用途。

本程式碼研究室不會說明如何使用 Cloud Run，相關內容請參閱 [Cloud Run 說明文件](#)。本節的目標是讓您熟悉如何將 App Engine 應用程式容器化，以便在 Cloud Run (或其他容器代管服務) 中執行。繼續操作前，請先瞭解幾件事，主要是使用者體驗會略有不同。

在本程式碼研究室中，您將瞭解如何建構及部署容器。內容如下：

- 使用 Jib 將應用程式容器化
- 從 App Engine 設定遷移
- 並視需要定義 Cloud Build 的建構步驟。

這項作業會涉及停用特定 App Engine 功能。如果不想採用這種做法，您還是可以[將應用程式保留在 App Engine 上](#)，同時升級至 Java 11/17 執行階段。

步驟 3 · 約 4 分鐘

3. 設定/準備工作

1. 設定專案

在本教學課程中，您將使用 [全新專案](#) 中的 [appengine-java-migration-samples](#) 存放區範例應用程式。確認專案具備有效的帳單帳戶。

如果您打算將現有的 App Engine 應用程式移至 Cloud Run，可以改用該應用程式來進行後續步驟。

執行下列指令，為專案啟用必要 API：

```
gcloud services enable artifactregistry.googleapis.com cloudbuild.googleapis.com
run.googleapis.com
```

2. 取得基準範例應用程式

在自己的電腦或 [Cloud Shell](#) 上複製範例應用程式，然後前往 [baseline](#) 資料夾。

這個範例是 Java 8 的 Servlet 型 Datastore 應用程式，適用於部署至 App Engine。請按照 README 中的操作說明，準備將這個應用程式部署至 App Engine。

3. (選用) 部署基準應用程式

如果您想在遷移至 Cloud Run 前，確認應用程式可在 App Engine 上運作，才需要執行下列步驟。

請參閱 README.md 中的步驟：

1. 安裝/重新熟悉 `gcloud` CLI
2. 使用 `gcloud init` 為專案初始化 `gcloud` CLI
3. 使用 `gcloud app create` 建立 App Engine 專案
4. 將範例應用程式部署至 App Engine

```
./mvnw package appengine:deploy -Dapp.projectId=$PROJECT_ID
```

5. 確認應用程式在 App Engine 上順利執行

4. 建立 Artifact Registry 存放區

將應用程式容器化後，您需要將映像檔推送至某處並儲存。在 Google Cloud 中，建議使用 [Artifact Registry](#) 執行這項操作。

使用 `gcloud` 建立名為 `migration` 的存放區，如下所示：

```
gcloud artifacts repositories create migration --repository-format=docker \
--description="Docker repository for the migrated app" \
--location="northamerica-northeast1"
```

請注意，這個存放區使用 `docker` 格式類型，但有 [多種存放區類型](#) 可供使用。

此時您已擁有基準 App Engine 應用程式，Google Cloud 專案也已準備好將其遷移至 Cloud Run。

4. 修改應用程式檔案

如果您的應用程式大量使用 App Engine 的舊版套裝組合服務、設定或其他僅限 App Engine 的功能，建議您在升級至新版執行階段時[繼續存取這些服務](#)。本程式碼研究室會說明應用程式的遷移路徑，這些應用程式已使用獨立服務，或可進行重構以使用獨立服務。

1. 升級至 Java 17

如果您的應用程式使用 Java 8，建議升級至較新的 LTS 候選版本 (例如 11 或 17)，以便取得安全性更新並使用新的語言功能。

首先，請更新 `pom.xml` 中的屬性，加入下列項目：

```
<properties>
  <java.version>17</java.version>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
```

這會將專案版本設為 17，通知編譯器外掛程式您想存取 Java 17 語言功能，並希望編譯的類別與 Java 17 JVM 相容。

2. 包括網路伺服器

在 App Engine 和 Cloud Run 之間遷移時，請考量兩者之間的差異。其中一項差異是，App Engine 的 Java 8 執行階段會為代管的應用程式提供及管理 Jetty 伺服器，但 Cloud Run 不會。我們會使用 Spring Boot 提供網路伺服器和 Servlet 容器。

新增下列相依項目：

```

<dependencies>
<!-- ... -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>2.6.6</version>
    <exclusions>
      <!-- Exclude the Tomcat dependency -->
      <exclusion>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-tomcat</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
  <!-- Use Jetty instead -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
    <version>2.6.6</version>
  </dependency>
<!-- ... -->
</dependencies>

```

Spring Boot 預設會嵌入 Tomcat 伺服器，但本範例會排除該構件，並繼續使用 Jetty，盡量減少遷移後預設行為的差異。我們也可以設定 Jetty 版本，與 [App Engine 提供的版本相符](#)。

```

<properties>
  <java.version>17</java.version>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <jetty.version>9.4.46.v20220331</jetty.version>
</properties>

```

3. 設定 Spring Boot

Spring Boot 能夠重複使用您的 Servlet，不必修改，但需要進行一些設定才能探索。

在 `com.example.appengine` 套件中建立下列 `MigratedServletApplication.java` 類別：


```
package com.example.appengine;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.web.servlet.ServletComponentScan;

@ServletComponentScan
@SpringBootApplication
@EnableAutoConfiguration
public class MigratedServletApplication {
    public static void main(String[] args) {
        SpringApplication.run(MigratedServletApplication.class, args);
    }
}
```

請注意，這包括 `@ServletComponentScan` 註解，該註解會 (預設在 [目前套件](#) 中) 尋找任何 `@WebServlets`，並如預期提供這些註解。

4. 將應用程式封裝為 JAR

雖然可以從 [WAR](#) 開始使用 [Jib 容器化應用程式](#)，但如果將應用程式封裝為可執行的 JAR，會更加容易。這不需要太多設定，特別是使用 Maven 做為建構工具的專案，因為 JAR 封裝是預設行為。

移除 `pom.xml` 檔案中的 `packaging` 標記：

```
<packaging>war</packaging>
```

接著，新增 `spring-boot-maven-plugin`：

```
<plugins>
<!-- ... -->
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
<version>2.6.6</version>
</plugin>
<!-- ... -->
</plugins>
```

5. 從 App Engine 設定、服務和依附元件遷移

如本程式碼研究室開頭所述，Cloud Run 和 App Engine 的設計宗旨是提供不同的使用者體驗。App Engine 提供的某些現成功能 (例如 [Cron](#) 和 [Task Queue](#) 服務) 必須手動重新建立，我們會在後續單元中詳細說明。

範例應用程式不會使用舊版套裝服務，但如果您的應用程式會使用，請參閱下列指南：

- [從服務套件遷移](#)，尋找合適的獨立服務。
- [將 XML 設定檔遷移至 YAML](#)：適用於遷移至 Java 11/17 執行階段，但仍使用 App Engine 的使用者。

由於您之後會部署至 Cloud Run，因此可以移除 `appengine-maven-plugin`：

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>appengine-maven-plugin</artifactId>
  <version>2.4.1</version>
  <configuration>
    <!-- can be set w/ -DprojectId=myProjectId on command line -->
    <projectId>${app.projectId}</projectId>
    <!-- set the GAE version or use "GCLLOUD_CONFIG" for an autogenerated GAE version -->
    <version>GCLLOUD_CONFIG</version>
  </configuration>
</plugin>
```

步驟 5 · 約 8 分鐘

5. 容器化應用程式

此時，您可以[直接從原始碼](#)手動將應用程式部署至 Cloud Run。這個絕佳選項會在幕後使用 Cloud Build，提供自動部署體驗。我們會在後續單元中詳細說明來源部署作業。

或者，如要進一步控管應用程式的部署方式，可以定義 `cloudbuild.yaml` 檔案，明確列出預期的建構步驟：

1. 定義 `cloudbuild.yaml` 檔案

在與 `pom.xml` 相同的層級建立下列 `cloudbuild.yaml` 檔案：

```
steps:
  # Test your build
  - name: maven:eclipse-temurin
    entrypoint: mvn
    args: ["test"]
  # Build with Jib
  - name: maven:eclipse-temurin
    entrypoint: mvn
    args: [ "compile", "com.google.cloud.tools:jib-maven-plugin:3.2.1:build", "-Dimage=northamerica-northeast1-docker.pkg.dev/PROJECT_ID/migration/visitors:jib" ]
  # Deploy to Cloud Run
  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    entrypoint: gcloud
    args: [ 'run', 'deploy', 'visitors', '--image', 'northamerica-northeast1-docker.pkg.dev/PROJECT_ID/migration/visitors:jib', '--region', 'northamerica-northeast1', '--allow-unauthenticated' ]
```

我們指示 Cloud Build 執行這些步驟後，系統會：

1. 使用 `./mvnw test` 執行測試
2. 使用 [Jib](#) 建構、推送映像檔並標記至 Artifact Registry
3. 使用 `gcloud run deploy` 將映像檔部署至 Cloud Run

請注意，`'visitors'` 會以所需服務名稱的形式提供給 Cloud Run。`--allow-unauthenticated` 旗標可讓使用者不必驗證就能造訪 Web 應用程式。請務必在 `cloudbuild.yaml` 檔案中，將 `PROJECT_ID` 替換為專案 ID。

接下來，請新增下列 IAM 政策繫結，允許 Cloud Build 服務帳戶存取 Artifact Registry：

```
export PROJECT_ID=$(gcloud config list --format 'value(core.project)')
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)"
)

gcloud projects add-iam-policy-binding $PROJECT_ID \
--member=serviceAccount:$PROJECT_NUMBER@cloudbuild.gserviceaccount.com \
--role=roles/run.admin \
--project=$PROJECT_ID
gcloud iam service-accounts add-iam-policy-binding $PROJECT_NUMBER-
compute@developer.gserviceaccount.com \
--member=serviceAccount:$PROJECT_NUMBER@cloudbuild.gserviceaccount.com \
--role roles/iam.serviceAccountUser --project=$PROJECT_ID
```

2. 執行建構程序

您已告知 Cloud Build 想要的建構步驟，現在可以一鍵部署。

執行下列指令：

```
gcloud builds submit
```

程序完成後，容器映像檔就會建構完成、儲存在 Artifact Registry 中，並部署至 Cloud Run。

完成本程式碼研究室後，您的應用程式應與 [java17-and-cloud-run/finish](#) 中的應用程式相同。

這樣就大功告成囉！您已成功將 Java 8 App Engine 應用程式遷移至 Java 17 和 Cloud Run，現在對切換及[選擇代管選項](#)所涉及的工作有更清楚的瞭解。

6. 摘要/清除

恭喜，您已升級、容器化及遷移應用程式，本教學課程到此結束！

接下來，您可以進一步瞭解 CI/CD 和軟體供應鏈安全功能，這些功能現在都能透過 Cloud Build 部署：

- [使用 Cloud Build 建立自訂建構步驟](#)
- [建立及管理自動建構觸發條件](#)
- [在 Cloud Build 管道中使用 On-Demand Scanning](#)

選用：清除及/或停用服務

如果您在本教學課程期間將範例應用程式部署至 App Engine，請記得[停用該應用程式](#)，以免產生費用。準備好進行下一個程式碼研究室時，可以重新啟用。停用 App Engine 應用程式後，應用程式不會收到任何流量，因此不會產生費用，但如果 [Datastore 用量](#) 超出 [免費配額](#)，系統仍會收費，因此請刪除足夠的資料，確保用量低於配額限制。

另一方面，如果您不打算繼續遷移，並想完全刪除所有內容，可以[刪除服務](#)或[完全關閉專案](#)。

7. 其他資源

App Engine 遷移模組程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

- [遷移選項](#)，可將 App Engine 服務從套裝組合中移除
- 為 Cloud Build 設定[建構觸發條件](#)
- 如要進一步瞭解如何遷移至 Java 11/17，請參閱[這篇文章](#)

線上資源

以下是可能與本教學課程相關的線上資源：

App Engine

- [App Engine 說明文件](#)
- App Engine [定價](#)和[配額](#)資訊
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)

其他雲端資訊

- [Google Cloud「永久免費」方案](#)
- [Google Cloud CLI \(`gcloud` CLI\)](#)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- 訂閱 [Google Cloud Tech](#)
- 訂閱 [Google Developers](#)

授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。
